

Funções

Uma função nada mais é do que um bloco de código (ou seja, declarações e outros comandos) que pode ser nomeado e chamado de dentro de um programa. Em outras palavras, uma função é uma sequência de comandos que recebe um nome e pode ser chamada de qualquer parte do programa, quantas vezes forem necessárias, durante a sua execução.

A linguagem C possui muitas funções já implementadas, e as temos utilizado constantemente. Um exemplo são as funções básicas de entrada e saída: **scanf()** e **printf()**. O programador não precisa saber o código contido dentro das funções de entrada e saída para utilizá-las. Basta saber seu nome e como utilizá-la.

Assim, a finalidade deste capítulo é apresentar os conceitos e detalhes necessários para um programador criar suas próprias funções dentro da linguagem C. Ao final, o leitor será capaz de:

- Declarar uma nova função.
- Definir os parâmetros de uma função.
- Definir o retorno de uma função.
- Fazer a passagem de parâmetros por valor para a função.
- Fazer a passagem de parâmetros por referência para a função.
- Utilizar o operador de seta.
- Utilizar o conceito de recursão em funções.

9.1 DEFINIÇÃO E ESTRUTURA BÁSICA

Duas são as principais razões para o uso de funções:

- Estruturação dos programas.
- Reutilização de código.

Por **estruturação dos programas** entende-se que agora o programa será construído a partir de pequenos blocos de código (isto é, funções), cada um deles com uma tarefa específica e bem-definida. Isso facilita a compreensão do programa.



Programas grandes e complexos são construídos bloco a bloco com a ajuda de funções.

Já por **reutilização de código** entende-se que uma função é escrita para realizar determinada tarefa. Pode-se definir, por exemplo, uma função para calcular o fatorial de determinado número. O código para essa função vai aparecer uma única vez em todo o programa, mas a função que calcula o fatorial poderá ser utilizada diversas vezes e em pontos diferentes do programa.



O uso de funções evita a cópia desnecessária de trechos de código que realizam a mesma tarefa, diminuindo assim o tamanho do programa e a ocorrência de erros.

9.1.1 Declarando uma função

Em linguagem C, a declaração de uma função pelo programador segue esta forma geral:

```
tipo_retornado nome_função (lista_de_parâmetros){
    sequência de declarações e comandos
}
```

O **nome_função** é como aquele trecho de código será conhecido dentro do programa. Para definir esse nome, valem basicamente as mesmas regras para se definir uma variável.

Local de declaração de uma função

Com relação ao local de declaração de uma função, ela deve ser definida ou declarada antes de ser utilizada, ou seja, antes da cláusula **main()**, como mostra o exemplo da Figura 9.1.

Exemplo: função declarada <i>antes</i> da cláusula main .	
01	#include <stdio.h>
02	#include <stdlib.h>
03	
04	int Square (int a){
05	return (a*a);
06	}
07	
08	int main(){
09	int n1,n2;
10	printf("Entre com um numero: ");
11	scanf("%d", &n1);
12	n2 = Square(n1);
13	printf("O seu quadrado vale: %d\n", n2);
14	system("pause");
15	return 0;
16	}

FIGURA 9.1

Pode-se também declarar uma função depois da cláusula **main()**. Nesse caso, é preciso declarar antes o protótipo da função:

```
tipo_retornado nome_função (lista_de_parâmetros);
```

O protótipo de uma função é uma declaração de função que omite o corpo mas especifica o seu nome, tipo de retorno e lista de parâmetros, como mostra o exemplo da Figura 9.2.

Exemplo: função declarada <i>depois</i> da cláusula <code>main</code> .	
01	<code>#include <stdio.h></code>
02	<code>#include <stdlib.h></code>
03	<code>//protótipo da função</code>
04	<code>int Square (int a);</code>
05	
06	<code>int main(){</code>
07	<code> int n1,n2;</code>
08	<code> printf("Entre com um numero: ");</code>
09	<code> scanf("%d", &n1);</code>
10	<code> n2 = Square(n1);</code>
11	<code> printf("O seu quadrado vale: %d\n", n2);</code>
12	<code> system("pause");</code>
13	<code> return 0;</code>
14	<code>}</code>
15	
16	<code>int Square (int a){</code>
17	<code> return (a*a);</code>
18	<code>}</code>

FIGURA 9.2



O protótipo de uma função não precisa incluir os nomes das variáveis passadas como parâmetros. Apenas os seus tipos já são suficientes.

A inclusão do nome de cada parâmetro no protótipo de uma função é tarefa opcional. Podemos declarar o seu protótipo apenas com os tipos dos parâmetros que serão passados para a função. Os nomes dos parâmetros são importantes apenas na implementação da função. Assim, ambos os protótipos a seguir são válidos para uma mesma função:

```
int Square (int a);
int Square (int );
```

Funcionamento de uma função

Independentemente de onde uma função seja declarada, seu funcionamento é basicamente o mesmo:

- O código do programa é executado até encontrar uma chamada de função.
- O programa é então interrompido temporariamente, e o fluxo do programa passa para a função chamada.

- Se houver parâmetros na função, os valores da chamada da função são copiados para os parâmetros no código da função.
- Os comandos da função são executados.
- Quando a função termina (seus comandos acabaram ou o comando **return** foi encontrado), o programa volta ao ponto em que foi interrompido para continuar sua execução normal.
- Se houver um comando **return**, o valor dele será copiado para a variável que foi escolhida para receber o retorno da função.

Na Figura 9.3, é possível ter uma boa representação de como uma chamada de função ocorre.

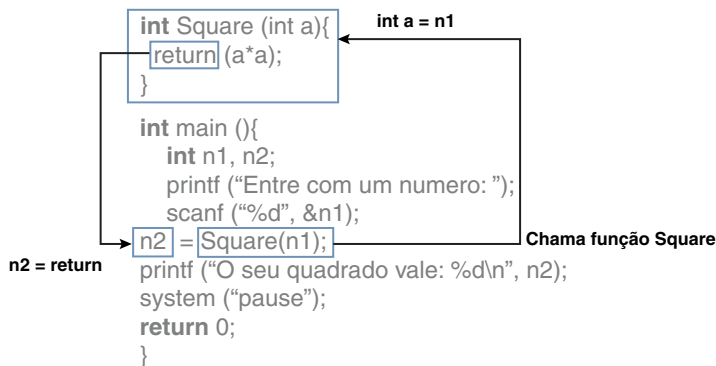


FIGURA 9.3

Nas seções seguintes, cada um dos itens que definem uma função serão apresentados em detalhes.

9.1.2 Parâmetros de uma função

Os parâmetros de uma função são o que o programador utiliza para passar a informação de um trecho de código para dentro da função. Basicamente, os parâmetros de uma função são uma lista de variáveis, separadas por vírgula, em que são especificados o tipo e o nome de cada variável passada para a função.



Por exemplo, a função **sqrt()** possui a seguinte lista de parâmetros: **double sqrt (double x);**

Declarando os parâmetros de uma função

Em linguagem C, a declaração dos parâmetros de uma função segue esta forma geral:

```

tipo_retornado nome_função (tipo nome1, tipo nome2, ... , tipo nomeN){
    sequência de declarações e comandos
}
  
```



Diferentemente do que acontece na declaração de variáveis, na qual muitas podem ser declaradas com o mesmo especificador de tipo, na declaração de parâmetros de uma função é necessário especificar o tipo de cada variável.

```
01 //Declaração CORRETA de parâmetros
02 int soma(int x, int y){
03     return x + y;
04 }
05
06 //Declaração ERRADA de parâmetros
07 int soma(int x, y){
08     return x + y;
09 }
```

Funções sem lista de parâmetros

Dependendo da função, ela pode não possuir nenhum parâmetro. Nesse caso, pode-se optar por duas soluções:

- Deixar a lista de parâmetros vazia: **void** imprime().
- Colocar **void** entre parênteses: **void** imprime(**void**).



Mesmo se não houver parâmetros na função, os parênteses ainda são necessários.

Apesar de as duas declarações estarem corretas, existe uma diferença entre elas. Na primeira declaração, não é especificado nenhum parâmetro, portanto a função pode ser chamada passando-se valores para ela. O compilador não vai verificar se a função é realmente chamada sem argumentos, e a função não conseguirá ter acesso a esses parâmetros. Já na segunda declaração, nenhum parâmetro é esperado. Nesse caso, o programa acusará um erro se o programador tentar passar um valor para essa função.



Colocar **void** na lista de parâmetros é diferente de não colocar nenhum parâmetro.

O exemplo da Figura 9.4 ilustra bem essa situação.

Exemplo: função sem parâmetros		
	Sem void	Com void
01	#include <stdio.h>	#include <stdio.h>
02	#include <stdlib.h>	#include <stdlib.h>
03		
04	void imprime(){	void imprime(void){
05	printf("Teste de funcao\n");	printf("Teste de funcao\n");
06	}	}
07		
08	int main(){	int main(){
09	imprime();	imprime();
10	imprime(5);	imprime(5);//ERRO
11	imprime(5,'a');	imprime(5,'a');//ERRO
12		
13	system("pause");	system("pause");
14	return 0;	return 0;
15	}	}

FIGURA 9.4

Os parâmetros das funções também estão sujeitos ao escopo das variáveis. O escopo é o conjunto de regras que determinam o uso e a validade de variáveis nas diversas partes do programa.



O parâmetro de uma função é uma variável local da função e, portanto, só pode ser acessado dentro da função.

9.1.3 Corpo da função

Pode-se dizer que o corpo de uma função é a sua alma. É no corpo de uma função que se define a tarefa que ela vai realizar quando for chamada.

Basicamente, o corpo da função é formado por:

- Sequência de declarações: variáveis, constantes, arrays etc.
- Sequência de comandos: comandos condicionais, de repetição, chamada de outras funções etc.

Para melhor entender o corpo da função, considere que todo programa possui ao menos uma função: a função **main()**. A função **main** é a função “principal” do programa, o “corpo” do programa. Note que, nos exemplos usados até agora, a função **main** é sempre do tipo **int**, e sempre retorna o valor 0:

```
int main() {
    sequência de declarações e comandos
    return 0;
}
```

Basicamente, é no corpo da função que as entradas (parâmetros) são processadas, as saídas são geradas ou outras ações são feitas. Além disso, a função **main** se encarrega de realizar a comunicação com o usuário, ou seja, é ela que realiza as operações de entrada e saída de dados (funções **scanf()** e **printf()**). Desse modo, tudo o que temos dentro de uma função **main()** pode ser feito em uma função desenvolvida pelo programador.



Tudo o que temos dentro da função **main** pode ser feito em uma função desenvolvida pelo programador.

Uma função é construída com o intuito de realizar uma tarefa específica e bem-definida. Por exemplo, uma função para calcular o fatorial deve ser construída de modo a receber determinado número como parâmetro e retornar (usando o comando **return**) o valor calculado. As operações de entrada e saída de dados (funções **scanf()** e **printf()**) devem ser feitas em quem chamou a função (por exemplo, na **main()**). Isso assegura que a função construída possa ser utilizada nas mais diversas aplicações, garantindo a sua generalidade.



De modo geral, evita-se fazer operações de leitura e escrita dentro de uma função.

Os exemplos das Figuras 9.5 e 9.6 ilustram bem essa situação. No primeiro exemplo temos o cálculo do fatorial realizado dentro da função main (Figura 9.5).

Exemplo: cálculo do fatorial dentro da função main()

```

01  #include <stdio.h>
02  #include <stdlib.h>
03
04  int main(){
05      printf("Digite um numero inteiro positivo:");
06      int x;
07      scanf("%d",&x);
08      int i,f = 1;
09      for (i=1; i<=x; i++)
10          f = f * i;
11
12      printf("O fatorial de %d eh: %d\\",x,f);
13      system("pause");
14      return 0;
15  }
```

FIGURA 9.5

Perceba que, nesse exemplo, não foi feito nada de diferente do que temos até o momento. Já no exemplo da Figura 9.6, uma função específica para o cálculo do fatorial foi construída.

Exemplo: cálculo do fatorial em uma função própria

```

01  #include <stdio.h>
02  #include <stdlib.h>
03
04  int fatorial (int n){
05      int i,f = 1;
06      for (i=1; i<=n; i++)
07          f = f * i;
08
09      return f;
10  }
11
12  int main(){
13      printf("Digite um numero inteiro positivo:");
14      int x;
15      scanf("%d",&x);
16      int fat = fatorial(x);
17      printf("O fatorial de %d eh: %d\\n",x,fat);
18
19      system("pause");
20      return 0;
21  }
```

FIGURA 9.6

Note que, dentro da função responsável pelo cálculo do fatorial, apenas o trecho do código responsável pelo cálculo do fatorial está presente. As operações de entrada e saída de dados (funções **scanf()** e **printf()**) são feitas em quem chamou a função **fatorial()**, ou seja, na função **main()**.



Operações de leitura e escrita não são proibidas dentro de uma função. Apenas não devem ser usadas se esse não for o foco da função.

Uma função deve conter apenas o trecho de código responsável por fazer aquilo que é o objetivo da função. Isso não impede que operações de leitura e escrita sejam utilizadas dentro dela. Elas só não devem ser usadas quando os valores podem ser passados para a função por meio dos parâmetros.

Na Figura 9.7 temos um exemplo de função que realiza operações de leitura e escrita.

Exemplo: função contendo operações de leitura e escrita

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  int menu(){
04      int i;
05      do {
06          printf("Escolha uma opção:\n");
07          printf("(1) Opcao 1\n");
08          printf("(2) Opcao 2\n");
09          printf("(3) Opcao 3\n");
10          scanf("%d", &i);
11      } while ((i < 1) || (i > 3));
12
13      return i;
14  }
15
16  int main(){
17      int op = menu();
18      printf("Vc escolheu a Opcao %d.\n",op);
19      system("pause");
20      return 0;
21  }

```

FIGURA 9.7

Nessa função, um menu de opções é apresentado ao usuário, que tem de escolher entre uma delas. A função se encarrega de verificar se a opção digitada é válida e, caso não seja, solicitar uma nova opção ao usuário.

9.1.4 Retorno da função

O retorno da função é a maneira como uma função devolve o resultado (se ele existir) da sua execução para quem a chamou. Nas seções anteriores, vimos que uma função segue esta forma geral:

```

tipo_retornado nome_função (lista_de_parâmetros){
    sequência de declarações e comandos
}

```

A expressão **tipo_retornado** estabelece o tipo de valor que a função vai devolver para quem a chamar. Uma função pode retornar qualquer tipo válido na linguagem C:

- Tipos básicos predefinidos: `int`, `char`, `float`, `double`, **`void`** e ponteiros.
- Tipos definidos pelo programador: `struct`, `array` (indiretamente) etc.

Funções sem retorno de valor



Uma função também pode NÃO retornar um valor. Para isso, basta colocar o tipo **`void`** como valor retornado.

O tipo **`void`** é conhecido como tipo *vazio*. Uma função declarada com o tipo **`void`** vai apenas executar um conjunto de comando e não devolverá nenhum valor para quem a chamar. Veja o exemplo da Figura 9.8.

Exemplo: função com tipo void

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  void imprime(int n){
04      int i;
05      for (i=1; i<=n; i++){
06          printf("Linha %d \n",i);
07      }
08
09  int main(){
10      imprime(5);
11
12      system("pause");
13      return 0;
14  }

```

FIGURA 9.8

Nesse exemplo, a função **`imprime`** apenas imprimirá uma mensagem na tela **`n`** vezes. Não há o que devolver para a função **`main()`**. Portanto, podemos declará-la como **`void`**.



Para executar uma função do tipo **`void`**, basta colocar no código onde a função será chamada o nome da função e seus parâmetros.

Funções com retorno de valor

Se a função não for do tipo **`void`**, ela deverá retornar um valor. O comando **`return`** é utilizado para retornar esse valor para o programa:

```
return expressão;
```



A expressão da cláusula **return** tem de ser compatível com o tipo de retorno declarado para a função.

A *expressão* do comando **return** consiste em qualquer constante, variável ou expressão aritmética que o programador deseje retornar para o trecho do programa que chamou a função. Essa expressão pode até mesmo ser uma outra função, como a função **sqrt()**:

```
return sqrt(x);
```



Para executar uma função que tenha o comando **return**, basta atribuir a chamada da função (nome da função e seus parâmetros) a uma variável compatível com o tipo do retorno.

O exemplo da Figura 9.9 mostra uma função que recebe dois parâmetros inteiros e retorna a sua soma para a função **main()**.

Exemplo: função com retorno

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  int soma(int x, int y){
04      return x + y;
05  }
06
07  int main(){
08      int a,b,c;
09      printf("Digite a: ");
10      scanf("%d", &a);
11      printf("Digite b: ");
12      scanf("%d", &b);
13      printf("Soma = %d\n",soma(a,b));
14      system("pause");
15      return 0;
16  }
```

FIGURA 9.9

Note, nesse exemplo, que a chamada da função foi feita dentro da função **printf()**. Isso é possível porque a função retorna um valor inteiro (**x + y**) e a função **printf()** espera imprimir um valor inteiro (**%d**).



Uma função pode ter mais de uma declaração **return**.

O uso de vários comandos **return** é útil quando o retorno da função está relacionado a determinada condição dentro dela. Veja o exemplo da Figura 9.10.

Exemplo: função com vários return

```

01 int maior(int x, int y){
02     if(x > y)
03         return x;
04     else
05         return y;
06 }

```

FIGURA 9.10

Nesse exemplo, a função será executada, e, dependendo dos valores de x e y , uma das cláusulas **return** será executada. No entanto, é conveniente limitar as funções a usar somente um comando **return**. O uso de vários comandos **return**, especialmente em funções grandes e complexas, aumenta a dificuldade de se compreender o que realmente está sendo feito pela função. Na maioria dos casos, pode-se reescrever uma função para que ela use somente um comando **return**, como é mostrado na Figura 9.11.

Exemplo: substituindo os vários return da função

```

01 int maior(int x, int y){
02     int z;
03     if(x > y)
04         z = x;
05     else
06         z = y;
07     return z;
08 }

```

FIGURA 9.11

Nesse exemplo, os vários comandos **return** foram substituídos por uma variável que será retornada no final da função.



Quando se chega a um comando **return**, a função é encerrada imediatamente.

O comando **return** é utilizado para retornar um valor para o programa. No entanto, esse comando também é usado para terminar a execução de uma função, similar ao comando **break** em um laço ou **switch** (Figura 9.12).

Exemplo: finalizando a função com return

```

01 int maior(int x, int y){
02     if(x > y)
03         return x;
04     else
05         return y;
06     printf("Fim da funcao\n");
07 }

```

FIGURA 9.12

Nesse exemplo, a função vai terminar quando um dos comandos **return** for executado. A mensagem “Fim da funcao” jamais será impressa na tela porque seu comando se encontra depois do comando **return**. Nesse caso, a função **printf()** será ignorada.



O comando **return** pode ser usado sem um valor associado a ele para terminar uma função do tipo **void**.

```

01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <math.h>
04 void imprime_log(float x){
05     if(x <= 0)
06         return;//termina a função
07     printf("Log = %f\n",log(x));
08 }
09 int main(){
10     float x;
11     printf("Digite x: ");
12     scanf("%f", &f);
13     imprime_log(x);
14     system("pause");
15     return 0;
16 }
```

Na função contida no exemplo anterior, se o valor de x é negativo ou zero, o comando **return** faz com que a função termine antes que o comando **printf()** seja executado, mas nenhum valor é retornado.



O valor retornado por uma função não pode ser um array.

Lembre-se: a linguagem C não suporta a atribuição de um array para outro. Por esse motivo, não se pode ter um array como retorno de uma função.



É possível retornar um array indiretamente, desde que ele faça parte de uma estrutura.

```

01 #include <stdio.h>
02 #include <stdlib.h>
03
04 struct vetor{
05     int v[5];
06 };
07
08 struct vetor retorna_array(){
09     struct vetor v = {1,2,3,4,5};
10     return v;
11 }
12
13 int main(){
14     int i;
15     struct vetor vet = retorna_array();
16     for (i=0; i<5; i++)
17         printf("Valores: %d \n",vet.v[i]);
18     system("pause");
19     return 0;
20 }
```

A linguagem C não suporta a atribuição de um array para outro, mas permite a atribuição entre estruturas. Isto faz com que os conteúdos das variáveis contidas dentro de uma estrutura sejam copiados para outra. Desse modo, é possível retornar um array desde que ele esteja dentro de uma estrutura.

9.2 TIPOS DE PASSAGEM DE PARÂMETROS

Já vimos que, na linguagem C, os parâmetros de uma função são o mecanismo que o programador utiliza para passar a informação de um trecho de código para dentro da função. Mas existem dois tipos de passagem de parâmetro: passagem **por valor** e **por referência**.

Nas seções seguintes, cada um dos tipos de passagem de parâmetros será explicado em detalhes.

9.2.1 Passagem por valor

Na linguagem C, os argumentos para uma função são sempre passados por valor (by value), ou seja, uma cópia do dado é feita e passada para a função. Esse tipo de passagem de parâmetro é o padrão para todos os tipos básicos predefinidos (**int**, **char**, **float** e **double**) e estruturas definidas pelo programador (**struct**).

	Mesmo que o valor de uma variável mude dentro da função, nada acontece com o valor de fora da função.
<pre> 01 include <stdio.h> 02 include <stdlib.h> 03 04 void soma_mais_um(int n){ 05 n = n + 1; 06 printf("Dentro da funcao: x = %d\n",n); 07 } 08 09 int main(){ 10 int x = 5; 11 printf("Antes da funcao: x = %d\n",x); 12 soma_mais_um(x); 13 printf("Depois da funcao: x = %d\n",x); 14 system("pause"); 15 return 0; 16 }</pre>	
Saída	<pre> Antes da funcao: x = 5 Dentro da funcao: x = 6 Depois da funcao: x = 5</pre>

Nesse exemplo, no momento em que a função **soma_mais_um()** é chamada, o valor de *x* é **copiado** para o parâmetro *n* da função. O parâmetro *n* é uma variável local da função. Então, tudo o que acontecer com ele (*n*) não se reflete no valor **original** da

variável x . Quando a função termina, a variável n é destruída e seu valor é descartado. O fluxo do programa é devolvido ao ponto em que a função foi inicialmente chamada, onde a variável x mantém o seu valor **original**.



Na passagem de parâmetros por valor, quaisquer modificações que a função fizer nos parâmetros existem apenas dentro da própria função.

9.2.2 Passagem por referência

Na passagem de parâmetros por valor, as funções não podem modificar o valor original de uma variável passada para a função. Mas existem casos em que é necessário que toda modificação feita nos valores dos parâmetros dentro da função seja repassada para quem a chamou. Um exemplo bastante simples disso é a função **scanf()**: sempre que desejamos ler algo do teclado, passamos para a função **scanf()** o nome da variável onde o dado será armazenado. Essa variável tem seu valor modificado dentro da função **scanf()**, e seu valor pode ser acessado no programa principal.



A função **scanf()** é um exemplo bastante simples de função que altera o valor de uma variável recebida por parâmetro, e essa mudança se reflete na variável fora da função.

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 int main(){
04     int x = 5;
05     printf("Antes do scanf: x = %d\n",x);
06     printf("Digite um numero: ");
07     scanf("%d",&x);
08     printf("Depois do scanf: x = %d\n",x);
09     system("pause");
10     return 0;
11 }
```

Quando se quer que o valor da variável mude dentro da função e essa mudança se reflita fora da função, usam-se passagens de parâmetros **por referência**.



Na passagem de parâmetros por referência não se passam para a função os valores das variáveis, mas os *endereços das variáveis na memória*.

Na passagem de parâmetros por referência, o que é enviado para a função é o endereço de memória onde a variável está armazenada, e não uma simples cópia de seu valor. Assim, utilizando o endereço da variável na memória, qualquer alteração que a variável sofra dentro da função será também refletida fora da função.



Para passar um parâmetro por referência, usa-se o operador **"&"** na frente do nome do parâmetro durante a declaração da função.

Para passar para a função um parâmetro por referência, a função precisa usar ponteiros. Um ponteiro é um tipo especial de variável que armazena um endereço de memória, da mesma maneira como uma variável armazena um valor. Mais detalhes sobre o uso de ponteiros serão apresentados no capítulo seguinte.

O exemplo da Figura 9.13 mostra a mesma função declarada usando a passagem de parâmetro de valor e por referência.

Exemplo: passagem por valor e referência		
	Por valor	Por referência
01	void soma_mais_um(int n){	void soma_mais_um(int *n){
02		
03		
04		

FIGURA 9.13

Note, nesse exemplo, que a diferença entre os dois tipos de passagem de parâmetro é o uso do operador “*” na passagem por referência. Consequentemente, toda vez que a variável passada por referência for usada dentro da função, o operador “*” deverá ser usado na frente do nome da variável.



Na chamada da função, é necessário utilizar o operador “&” na frente do nome da variável que será passada por referência para a função.

Lembre-se do exemplo da função **scanf()**. A função **scanf()** é um exemplo de função que altera o valor de uma variável, e essa mudança se reflete fora da função. Quando chamamos a função **scanf()**, é necessário colocar o operador “&” na frente do nome da variável que será lida do teclado. O mesmo vale para outras funções que usam passagem de parâmetro por referência.



Na passagem de uma variável por referência é necessário usar o operador “*” sempre que se desejar acessar o conteúdo da variável dentro da função.

```
01 include <stdio.h>
02 include <stdlib.h>
03
04 void soma_mais_um(int *n){
05     *n = *n + 1;
06     printf("Dentro da funcao: x = %d\n",*n);
07 }
08
09 int main(){
10     int x = 5;
11     printf("Antes da funcao: x = %d\n",x);
12     soma_mais_um(&x);
13     printf("Depois da funcao: x = %d\n",x);
14     system("pause");
15     return 0;
16 }
```

Saída	Antes da funcao: x = 5 Dentro da funcao: x = 6 Depois da funcao: x = 6
-------	--

Nesse exemplo, no momento em que a função **soma_mais_um** é chamada, o endereço de *x* (&*x*) é **copiado** para o parâmetro *n* da função. O parâmetro *n* é um ponteiro dentro da função que guarda o endereço de onde o valor de *x* está guardado fora da função. Sempre que alteramos o valor de **n* (conteúdo da posição de memória guardada, ou seja, *x*), o valor de *x* fora da função também é modificado.

Na Figura 9.14 temos outro exemplo que mostra a mesma função declarada usando a passagem de parâmetro de valor e por referência.

Exemplo: passagem por valor e referência		
	Por valor	Por referência
01	#include <stdio.h>	#include <stdio.h>
02	#include <stdlib.h>	#include <stdlib.h>
03		
04	void Troca(int a, int b){	void Troca(int *a, int *b){
05	int temp;	int temp;
06	temp = a;	temp = *a;
07	a = b;	*a = *b;
08	b = temp;	*b = temp;
09	printf("Dentro: %d e %d\n",a,b);	printf("Dentro: %d e %d\n",*a,*b);
10	}	}
11		
12	int main(){	int main(){
13	int x = 2;	int x = 2;
14	int y = 3;	int y = 3;
15	printf("Antes: %d e %d\n",x,y);	printf("Antes: %d e %d\n",x,y);
16	Troca(x,y);	Troca(&x,&y);
17	printf("Depois: %d e %d\n",x,y);	printf("Depois: %d e %d\n",x,y);
18	system("pause");	system("pause");
19	return 0;	return 0;
20	}	}
	Saída Antes: 2 e 3 Dentro: 3 e 2 Depois: 2 e 3	Saída Antes: 2 e 3 Dentro: 3 e 2 Depois: 3 e 2

FIGURA 9.14

9.2.3 Passagem de arrays como parâmetros

Para utilizar arrays como parâmetros de funções, alguns cuidados simples são necessários. Além do parâmetro do array que será utilizado na função, é necessário declarar um segundo parâmetro (em geral, uma variável inteira) para passar para a função o tamanho do array **separadamente**.

 Arrays são sempre passados *por referência* para uma função.

Quando passamos um array por parâmetro, independentemente do seu tipo, o que é de fato passado para a função é o endereço do primeiro elemento do array.



A passagem de arrays por referência evita a cópia desnecessária de grande quantidade de dados para outras áreas de memória durante a chamada da função, o que afetaria o desempenho do programa.

Na passagem de um array como parâmetro de uma função podemos declarar a função usando uma das maneiras mostradas a seguir, todas equivalentes:

```
void imprime (int *m, int n);
void imprime (int m[], int n);
void imprime (int m[5], int n);
```



Mesmo especificando o tamanho de um array no seu parâmetro da função, a semântica é a mesma das outras declarações, pois não existe checagem dos limites do array em tempo de compilação.

O exemplo da Figura 9.15 mostra como um array de uma única dimensão pode ser passado como parâmetro para uma função.

Exemplo: passagem de array como parâmetro

```
01  #include <stdio.h>
02  #include <stdlib.h>
03
04  void imprime (int *n, int m){
05      int i;
06      for (i=0; i<m;i++)
07          printf("%d \n", n[i]);
08  }
09
10  int main(){
11      int v[5] = {1,2,3,4,5};
12      imprime(v,5);
13      system("pause");
14      return 0;
15  }
```

FIGURA 9.15

Note, nesse exemplo, que apenas o nome do array é passado para a função, sem colchetes e sem índice. Isso significa que estamos passando o array inteiro. Se usássemos o colchete, estaríamos passando o valor de uma posição do array e não o seu endereço, o que resultaria em erro. Note também que não precisamos utilizar o operador “&”. Isso acontece porque o nome do array é o seu endereço na memória. Diferentemente de outras variáveis passadas por referência, dentro da função não precisamos usar o operador “*” para acessar o conteúdo de determinada posição do array. Podemos continuar usando os colchetes e o seu índice. Isso ocorre devido às propriedades em comum dos arrays e ponteiros, que serão apresentadas no capítulo seguinte.



Na chamada da função, passamos para ela somente o nome do array, sem os colchetes, sem o índice e sem o operador “&”: o programa “já sabe” que um array será enviado, pois isso já foi definido no protótipo da função. Dentro da função, não precisamos usar o operador “*” para acessar o conteúdo de uma posição do array, podemos continuar usando os colchetes.

Vimos que, para arrays, não é necessário especificar o número de elementos para a função no parâmetro do array:

```
void imprime (int *m, int n);
void imprime (int m[], int n);
```



Arrays com mais de uma dimensão (por exemplo, matrizes) precisam da informação do tamanho das dimensões extras.

Para arrays com mais de uma dimensão é necessário especificar o tamanho de todas as dimensões, exceto a primeira. Sendo assim, uma declaração possível para uma função que receba uma matriz de quatro linhas e cinco colunas seria a apresentada a seguir:

```
void imprime (int m[][5], int n);
```

A declaração de arrays com uma dimensão e com mais de uma dimensão é diferente porque, na passagem de um array para uma função, o compilador precisa saber o tamanho de cada elemento, não o número de elementos.



Um array bidimensional pode ser entendido como um array de arrays.

Para a linguagem C, um array bidimensional pode ser entendido como um array de arrays. Sendo assim, o seguinte array

```
int m[4][5];
```

pode ser entendido como um array de quatro elementos, em que cada elemento é um array de cinco posições inteiras. Logo, o compilador precisa saber o tamanho de um dos elementos (por exemplo, o número de colunas da matriz) no momento da declaração da função:

```
void imprime (int m[][5], int n);
```

Nessa notação, informamos ao compilador que estamos passando um array, onde cada elemento dele é outro array de cinco posições inteiras. Nesse caso, o array terá sempre cinco colunas, mas poderá ter quantas linhas quiser (parâmetro **n**).

Isso é necessário para que o programa saiba que o array possui mais de uma dimensão e mantenha a notação de um conjunto de colchetes por dimensão.

O exemplo da Figura 9.16 mostra como um array de duas dimensões pode ser passado como parâmetro para uma função.

Exemplo: passagem de matriz como parâmetro

```

01  #include <stdio.h>
02  #include <stdlib.h>
03
04  void imprime _matriz(int m[][2], int n){
05      int i,j;
06      for (i=0; i<n;i++)
07          for (j=0; j< 2;j++)
08              printf("%d \n", m[i][j]);
09  }
10
11  int main(){
12      int mat[3][2] = {{1,2},{3,4},{5,6}};
13      imprime _matriz(mat,3);
14      system("pause");
15      return 0;
16  }

```

FIGURA 9.16

As notações a seguir funcionam para arrays com mais de uma dimensão. Mas o array é tratado como se tivesse apenas uma dimensão dentro da função

```

void imprime (int *m, int n);
void imprime (int m[], int n);

```

O exemplo da Figura 9.17 mostra como um array de duas dimensões pode ser passado como um array de uma única dimensão para uma função.

Exemplo: matriz como array de uma dimensão

```

01  #include <stdio.h>
02  #include <stdlib.h>
03
04  void imprime _matriz(int *m, int n){
05      int i;
06      for (i=0; i<n;i++)
07          printf("%d \n", m[i]);
08  }
09
10  int main(){
11      int mat[3][2] = {{1,2},{3,4},{5,6}};
12      imprime _matriz(&mat[0][0],6);
13      system("pause");
14      return 0;
15  }

```

FIGURA 9.17

Note que, nesse exemplo, em vez de passarmos o nome do array, passamos o endereço do primeiro elemento (&mat[0][0]). Isso faz com que percamos a notação de dois colchetes para a matriz e ela seja tratada como se tivesse apenas uma dimensão.

9.2.4 Passagem de estruturas como parâmetros

Vimos anteriormente que uma estrutura pode ser vista como um conjunto de variáveis sob um mesmo nome, ou, em outras palavras, a estrutura é uma variável que contém dentro de si outras variáveis. Sendo assim, uma estrutura pode ser passada como parâmetro para uma função de duas formas distintas:

- Toda a estrutura.
- Apenas determinados campos da estrutura.



As regras de passagem de uma estrutura como parâmetro para uma função são as mesmas usadas para passar uma união ou uma enumeração como parâmetro.

Passagem de uma estrutura por valor

Para passar uma estrutura como parâmetro de uma função, basta declarar na lista de parâmetros um parâmetro com o mesmo tipo da estrutura. Dessa forma, teremos acesso a todos os campos da estrutura dentro da função, como mostra o exemplo da Figura 9.18.

Exemplo: estrutura passada por valor

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  struct ponto {
04      int x, y;
05  };
06  void imprime(struct ponto p){
07      printf("x = %d\n",p.x);
08      printf("y = %d\n",p.y);
09  }
10  int main(){
11      struct ponto p1 = {10,20};
12      imprime(p1);
13
14      system("pause");
15      return 0;
16  }
```

FIGURA 9.18

Passagem de um campo da estrutura por valor

Dependendo da aplicação, pode ser que não seja necessário passar todos os valores da estrutura para a função. Nesse caso, a função é declarada sem levar em conta a estrutura nos seus parâmetros. Mas é necessário que o parâmetro da função seja compatível com o campo da função que será passado como parâmetro, como mostra o exemplo da Figura 9.19.

Exemplo: campo da estrutura passado por valor

```

01 #include <stdio.h>
02 #include <stdlib.h>
03 struct ponto {
04     int x, y;
05 };
06 void imprime_valor(int n){
07     printf("Valor = %d\n",n);
08 }
09 int main(){
10     struct ponto p1 = {10,20};
11     imprime_valor(p1.x);
12     imprime_valor(p1.y);
13     system("pause");
14     return 0;
15 }

```

FIGURA 9.19

Passagem de um campo da estrutura por referência

Como visto anteriormente, podemos passar um único campo da estrutura por valor para a função. Isso significa que também podemos passar esse mesmo campo por referência. Nesse caso, o parâmetro da função deverá novamente ser compatível com o campo da função que será passado como parâmetro, adicionado do operador “*”, como mostra o exemplo da Figura 9.20.

Exemplo: campo da estrutura passada por referência

```

01 #include <stdio.h>
02 #include <stdlib.h>
03 struct ponto {
04     int x, y;
05 };
06 void soma_imprime_valor(int *n){
07     *n = *n + 1;
08     printf("Valor = %d\n",*n);
09 }
10 int main(){
11     struct ponto p1 = {10,20};
12     soma_imprime_valor(&p1.x);
13     soma_imprime_valor(&p1.y);
14     system("pause");
15     return 0;
16 }

```

FIGURA 9.20

De fato, como se pode ver, para a passagem de um campo da estrutura por referência valem todas as regras vistas para a passagem de parâmetros por referência.

Passagem de uma estrutura por referência

Vimos anteriormente que, para passar um parâmetro por referência, usa-se o operador “*” na frente do nome do parâmetro durante a declaração da função. Isso também

é válido para uma estrutura, mas alguns cuidados devem ser tomados ao acessar seus campos dentro da função. Para acessar o valor de um campo de uma estrutura passada por referência, devemos seguir este conjunto de passos:

1. Utilizar o operador “*” na frente do nome da variável que representa a estrutura.
2. Colocar o operador “*” e o nome da variável entre parênteses ().
3. Por fim, acessar o campo da estrutura utilizando o operador ponto “.”.

O exemplo da Figura 9.21 mostra como os campos de uma estrutura passada por referência devem ser acessados.

Exemplo: estrutura passada por referência	
01	<code>#include <stdio.h></code>
02	<code>#include <stdlib.h></code>
03	<code>struct ponto {</code>
04	<code> int x, y;</code>
05	<code>};</code>
06	<code>void atribui(struct ponto *p){</code>
07	<code> (*p).x = 10;</code>
08	<code> (*p).y = 20;</code>
09	<code>}</code>
10	<code>int main(){</code>
11	<code> struct ponto p1;</code>
12	<code> atribui(&p1);</code>
13	<code> printf("x = %d\n",p1.x);</code>
14	<code> printf("y = %d\n",p1.y);</code>
15	<code> system("pause");</code>
16	<code> return 0;</code>
17	<code>}</code>

FIGURA 9.21

Note, nesse exemplo, que a função **atribui** recebe uma **struct ponto** por referência, “*p”. Para acessar qualquer um dos seus campos (**x** ou **y**), é necessário utilizar o operador “*” na frente do nome da variável que representa a estrutura, “*p”, e em seguida colocar o operador “*” e o nome da variável entre parênteses, “(*p)”. Somente depois de feito isso é que podemos acessar um dos campos da estrutura com o operador ponto “.” (linhas 7 e 8).



Ao acessar uma estrutura passada por referência, não podemos esquecer de colocar os parênteses antes de acessar o seu campo.

O uso dos parênteses serve para diferenciar o que foi passado por referência do que é ponteiro. Um ponteiro é um tipo especial de variável que armazena um endereço de memória, da mesma maneira como uma variável armazena um valor (mais detalhes sobre o uso de ponteiros serão apresentados no capítulo seguinte). A expressão

`(*p).x`

indica que a variável **p** é na verdade o ponteiro, ou melhor, a variável que foi passada por referência. Isso ocorre porque o asterisco está junto de **p** e isolado de **x** por meio dos parênteses. Já as notações a seguir são equivalentes

```
*p.x
*(p.x)
```

e ambas indicam que a variável **x** é na verdade o ponteiro, e não **p**. Isso ocorre porque o operador ponto “.” tem prioridade e é executado primeiro. Logo, o operador asterisco “*” atuará sobre o campo da estrutura, e não sobre a variável da estrutura.

9.2.5 Operador seta

De modo geral, uma estrutura é sempre passada por valor para uma função. Mas ela também pode ser passada por referência sempre que desejarmos alterar algum dos valores de seus campos.

Durante o estudo dos tipos definidos pelo programador, vimos que o operador “.” (ponto) era utilizado para acessar os campos de uma estrutura. Se essa estrutura for passada por referência para uma função, será necessário usar os operadores “*” e “.” para acessar os valores originais dos campos da estrutura.

- Operador “*”: acessa o conteúdo da posição de memória (valor da variável fora da função) dentro da função.
- Operador “.”: acessa os campos de uma estrutura.



O operador **seta** (->) substitui o uso conjunto dos operadores “*” e “.” no acesso ao campo de uma estrutura passada por referência para uma função.

O operador **seta** (->) é utilizado quando uma referência para uma estrutura (struct) é passada para uma função. Ele permite acessar o valor do campo da estrutura fora da função sem utilizar o operador “*”. O exemplo da Figura 9.22 mostra como os campos de uma estrutura passada por referência podem ser acessados com ou sem o uso do operador seta “->”.

Exemplo: passagem de estrutura por referência		
	Sem operador seta	Com operador seta
01	struct ponto {	struct ponto {
02	int x, y;	int x, y;
03	};	};
04		
05	void atribui(struct ponto *p){	void atribui(struct ponto *p){
06	(*p).x = 10;	p->x = 10;
07	(*p).y = 20;	p->y = 20;
08	}	}

FIGURA 9.22



O operador **seta** “->” também pode ser usado para acessar o valor do campo de uma união ou uma enumeração passada por referência para a função.

9.3 RECURSÃO

Na linguagem C, uma função pode chamar outra função, como mostra o exemplo da Figura 9.23. A função **main()** pode chamar qualquer função, seja ela da biblioteca da linguagem (como a função **printf()**) ou definida pelo programador (função **imprime()**). O mesmo vale para as funções definidas pelo programador: elas também podem chamar outras funções, sejam elas da biblioteca da linguagem ou definidas pelo programador.

Exemplo: chamadas de função

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  void imprime(int n){
04      int i;
05      for (i=1; i<=n; i++)
06          printf("Linha %d \n",i);
07  }
08
09  int main(){
10      imprime(5);
11      printf("Fim do programa!\n");
12
13      system("pause");
14      return 0;
15  }
```

FIGURA 9.23

Uma função pode, inclusive, chamar a si própria. A isso chamamos recursividade, que é o processo de repetir alguma coisa de maneira similar. Uma função assim é chamada de *função recursiva*.



A recursão também é chamada de definição circular. Ela ocorre quando algo é definido em termos de si mesmo.

9.3.1 Solução recursiva

Para entender como funciona a recursão, imagine o seguinte problema:



Como esvaziar um vaso contendo três flores?

Para esvaziar um vaso contendo três flores, primeiro verificamos se o vaso está vazio. Se o vaso não está vazio, tiramos uma flor. Temos agora que esvaziar o vaso contendo duas flores.



Como esvaziar um vaso contendo duas flores?

Para esvaziar um vaso contendo duas flores, primeiro verificamos se o vaso está vazio. Se o vaso não está vazio, tiramos uma flor. Temos agora que esvaziar o vaso contendo uma flor.



Como esvaziar um vaso contendo uma flor?

Para esvaziar o vaso contendo uma flor, primeiro verificamos se o vaso está vazio. Se o vaso não está vazio, tiramos uma flor. Temos agora que esvaziar o vaso contendo zero flores.



Como esvaziar um vaso contendo zero flores?

Para esvaziar o vaso contendo zero flores, primeiro verificamos se o vaso está vazio. Como ele já está vazio, o processo termina.

Todo esse processo de remover flores é muito repetitivo. Vamos então generalizar a ideia.



Como esvaziar um vaso contendo N flores?

Solução: Para esvaziar um vaso contendo N flores, primeiro verificamos se o vaso está vazio. Se o vaso não está vazio, tiramos uma flor. Temos agora que esvaziar um vaso contendo $N - 1$ flores.

Essa solução para o problema do vaso de flores pode agora ser facilmente implementada, como mostra o exemplo da Figura 9.24. Perceba que definimos a função que esvazia o vaso, **esvaziarVaso()**, em termos de si mesma. Trata-se, portanto, de uma função recursiva.

Exemplo: função recursiva para esvaziar vaso de flores

```
01 void esvaziarVaso(int flores){
02     if(flores > 0){
03         //remove uma flor
04         esvaziarVaso(flores - 1);
05     }
06 }
```

FIGURA 9.24

Um exemplo clássico de função recursiva é o cálculo do fatorial de um número.



Como calcular o fatorial de 4 (definido como 4!)?

Solução: Para calcular o fatorial de 4, multiplica-se o número 4 pelo fatorial de 3 (definido como 3!). Nesse ponto, já é possível perceber que esse problema é muito semelhante ao do vaso de flores. Generalizando esse processo, temos que o fatorial de N é igual a N multiplicado pelo fatorial de $(N - 1)$, ou seja, $N! = N * (N - 1)!$. No caso do vaso de flores, o processo termina quando não há mais flores no vaso. No caso do fatorial, o processo termina quando atingimos o número zero. Nesse caso, o valor do fatorial de 0 (0!) é definido como igual a 1. Temos então que a função fatorial é definida matematicamente como:

$$\begin{aligned} 0! &= 1 \\ N! &= N * (N - 1)! \end{aligned}$$

Note novamente que o fatorial de N , $N!$, é definido em termos do fatorial de $N - 1$, $(N - 1)!$, ou seja, trata-se de uma definição circular do problema: precisamos saber o fatorial de um número para calcular o de outro. Esse processo continua até que se chegue a um caso mais simples e que é a base do cálculo do fatorial: o fatorial de zero, $0!$. Nesse momento, a recursão para porque temos um valor constante já definido para o fatorial. Na Figura 9.25, é possível ver a função recursiva para o cálculo do fatorial:

Exemplo: função recursiva para cálculo do fatorial	
01	<code>int fatorial (int n){</code>
02	<code> if(n == 0)</code>
03	<code> return 1;</code>
04	<code> else</code>
05	<code> return n*fatorial(n-1);</code>
06	<code>}</code>

FIGURA 9.25

9.3.2 Como funciona a recursividade

A ideia básica da recursão é **dividir e conquistar**:

- Divide-se um problema maior em um conjunto de problemas menores.
- Esses problemas menores são então resolvidos de forma independente.
- As soluções dos problemas menores são combinadas para gerar a solução final.

Essa lógica fica evidente no cálculo do fatorial. O fatorial de um número N é o produto de todos os números inteiros entre 1 e N . Por exemplo, o fatorial de 4 é $1 * 2 * 3 * 4$. Aplicando a ideia da recursão, temos que:

- O fatorial de 4 é definido em função do fatorial de 3.
- O fatorial de 3 é definido em função do fatorial de 2.

- O fatorial de 2 é definido em função do fatorial de 1.
- O fatorial de 1 é definido em função do fatorial de 0.
- O fatorial de 0 é definido como igual a 1.

Perceba que o cálculo do fatorial prossegue até chegarmos ao fatorial de 0, nosso **caso-base**, que é igual a 1.



Quando a função recursiva chega ao seu **caso-base**, ela para.

Vamos chamar essa primeira etapa do cálculo de **caminho de ida da recursão**, que é onde as chamadas da função são executadas até chegar ao caso-base. Quando a recursão para, é chegado o momento de fazer o **caminho de volta da recursão** que, basicamente, consiste em fazer o caminho inverso devolvendo o valor obtido para quem fez aquela chamada da função, e assim por diante, até chegarmos à primeira chamada da função, como mostra a Figura 9.26.

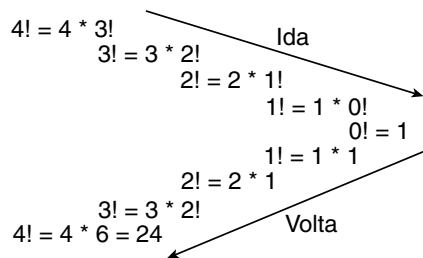


FIGURA 9.26

Os mesmos princípios de **caso-base**, **caminho de ida da recursão** e **caminho de volta da recursão**, presentes no exemplo matemático da recursão, estão também presentes na recursão de uma função computacional. Considere o exemplo da Figura 9.27.

Exemplo: calculando o fatorial

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  int fatorial (int n){
04      if(n == 0)
05          return 1;
06      else
07          return n*fatorial(n-1);
08  }
09  int main(){
10      int x;
11      x = fatorial(4);
12      printf("4! = %d\n",x);
13      system("pause");
14      return 0;
15  }
```

FIGURA 9.27

Sempre que fazemos uma chamada de função, seja ela qual for, o código atualmente em execução é pausado, e a função chamada é carregada na memória. Assim, a função `main()` é pausada quando chama a função `fatorial()`. Note que a função `fatorial()` também chama a si própria. Nesse caso, ela é pausada e carrega na memória uma cópia de si mesma, mas com valor de parâmetro diferente ($n - 1$). Esse processo se repete até que a função seja chamada com o valor de n igual a 0. Esse é o **caso-base** em que a função para de chamar a si mesma, como mostra a Figura 9.28.

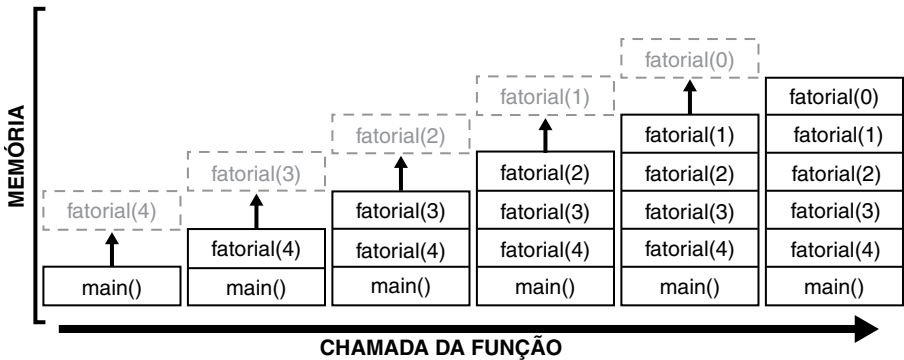


FIGURA 9.28

Uma vez que chegamos ao **caso-base**, é hora de fazer o **caminho de volta da recursão**. Sempre que uma função termina de executar, ela devolve (se existir) seu resultado para quem a chamou e, em seguida, é removida da memória. Assim, a chamada da função `fatorial(0)` devolverá o valor 1 para a função à chamada da função `fatorial(1)`, que vai devolver o valor $1 * 1$ para a chamada da função `fatorial(2)`, e assim por diante, como mostra a Figura 9.29.

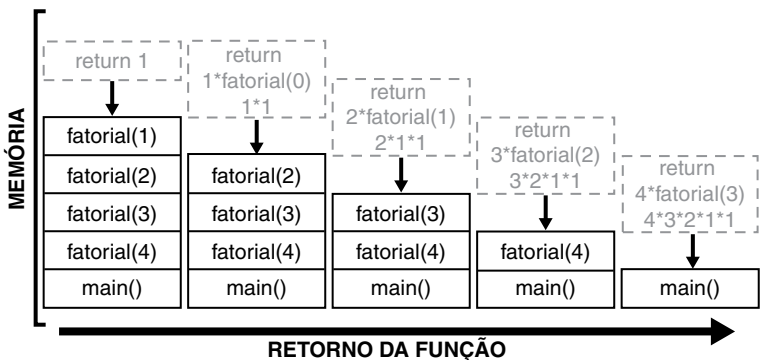


FIGURA 9.29

Saber identificar o **caso-base**, o **caminho de ida da recursão** e o **caminho de volta da recursão** torna a construção de uma função recursiva bastante simples.

9.3.3 Cuidados na implementação da recursividade

O exemplo da Figura 9.30 apresenta as funções com e sem recursão para o cálculo do fatorial.

Exemplo: fatorial		
	Com recursão	Sem recursão
01	<code>int fatorial (int n){</code>	<code>int fatorial (int n){</code>
02	<code>if(n == 0)</code>	<code>if(n == 0)</code>
03	<code>return 1;</code>	<code>return 1;</code>
04	<code>else</code>	<code>else {</code>
05	<code>return n*fatorial(n-1);</code>	<code>int i, f = 1;</code>
06	<code>}</code>	<code>for (i=2; i <= n;i++)</code>
07		<code>f = f * i;</code>
08		<code>return f;</code>
09		<code>}</code>
10		<code>}</code>

FIGURA 9.30

Em geral, as formas recursivas dos algoritmos são consideradas “mais enxutas” e “mais elegantes” do que suas formas iterativas. Isso facilita a interpretação do código. Porém, esses algoritmos apresentam maior dificuldade na detecção de erros e podem ser ineficientes.



Todo cuidado é pouco ao se fazerem funções recursivas, pois duas coisas devem ficar bem estabelecidas: o *critério de parada* e o *parâmetro da chamada recursiva*.

Durante a implementação de uma função recursiva temos de ter em mente duas coisas: o **critério de parada** e o **parâmetro da chamada recursiva**:

- **Critério de parada:** determina quando a função deve parar de chamar a si mesma. Se ele não existir, a função continuará executando até esgotar a memória do computador. No cálculo de fatorial, o critério de parada ocorre quando tentamos calcular o fatorial de zero: $0! = 1$.
- **Parâmetro da chamada recursiva:** quando chamamos a função dentro dela mesma, devemos sempre mudar o valor do parâmetro passado, de forma que a recursão chegue a um término. Se o valor do parâmetro for sempre o mesmo, a função continuará executando até esgotar a memória do computador. No cálculo de fatorial, a mudança no parâmetro da chamada recursiva ocorre quando definimos o fatorial de N em termos no fatorial de $(N - 1)$: $N! = N * (N - 1)!$.

O exemplo da Figura 9.31 deixa bem claros o **critério de parada** e o **parâmetro da chamada recursiva** na função recursiva implementada em linguagem C.

Exemplo: fatorial	
01	<code>int fatorial (int n){</code>
02	<code> if(n == 0) //critério de parada</code>
03	<code> return 1;</code>
04	<code> else //parâmetro do fatorial sempre muda</code>
05	<code> return n*fatorial(n - 1);</code>
06	<code>}</code>

FIGURA 9.31

Note que a implementação da função recursiva do fatorial em linguagem C segue exatamente o que foi definido matematicamente.



Algoritmos recursivos tendem a necessitar de mais tempo e/ou espaço do que algoritmos iterativos.

Sempre que chamamos uma função, é necessário espaço de memória para armazenar os parâmetros, variáveis locais e endereço de retorno da função. Em uma função recursiva, essas informações são armazenadas para cada chamada da recursão, sendo, portanto, a memória necessária para armazená-las proporcional ao número de chamadas da recursão. Por exemplo, para calcular o fatorial do número 4 são necessárias cinco chamadas da função fatorial.

Além disso, todas essas tarefas de alocar e liberar memória, copiar informações etc. envolvem tempo computacional, de modo que uma função recursiva gasta mais tempo que sua versão iterativa (sem recursão).

Outro exemplo clássico de recursão é a sequência de Fibonacci:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

A sequência de Fibonacci é definida como uma função recursiva utilizando a fórmula a seguir:

$$F(n) = \begin{cases} 0, & \text{se } n = 0 \\ 1, & \text{se } n = 1 \\ F(n - 1) + F(n - 2), & \text{outros casos} \end{cases}$$

O exemplo da Figura 9.32 apresenta as funções com e sem recursão para o cálculo da sequência de Fibonacci.

Exemplo: sequência de Fibonacci		
	Com recursão	Sem recursão
01	<code>int fibo(int n){</code>	<code>int fibo(int n){</code>
02	<code> if(n == 0 n == 1)</code>	<code> int i,t,c,a=0, b=1;</code>
03	<code> return n;</code>	<code> for(i=0;i<n;i++){</code>
04	<code> else</code>	<code> c = a + b;</code>
05	<code> return fibo(n-1) + fibo(n-2);</code>	<code> a = b;</code>
06	<code>}</code>	<code> b = c;</code>
07		<code> }</code>
08		<code> return a;</code>
09		<code>}</code>

FIGURA 9.32

Como se nota, a solução recursiva para a sequência de Fibonacci é muito elegante. Infelizmente, ela contém duas chamadas para si mesma. Isso significa que ela contém duas chamadas recursivas. Logo, sua elegância não significa eficiência, como se verifica na Figura 9.33.

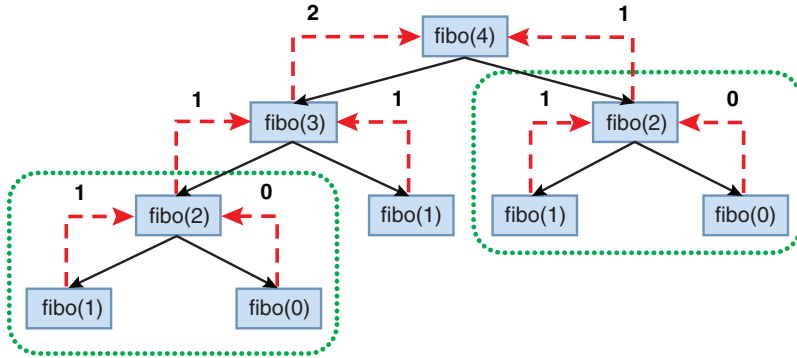


FIGURA 9.33

Nessa figura, as setas inteiras indicam quando uma nova chamada da função é realizada, enquanto as setas tracejadas indicam o processo inverso, ou seja, quando a função passa a devolver para quem a chamou o valor do comando **return**. O maior problema dessa solução recursiva está nas caixas pontilhadas. Neles, fica claro que parte do cálculo é realizada duas vezes pelas duas chamadas recursivas: **um grande desperdício de tempo e espaço!**

Se, em vez de calcularmos **fibo(4)**, quisermos calcular **fibo(5)**, teremos um desperdício ainda maior de tempo e espaço, como mostra a Figura 9.34.

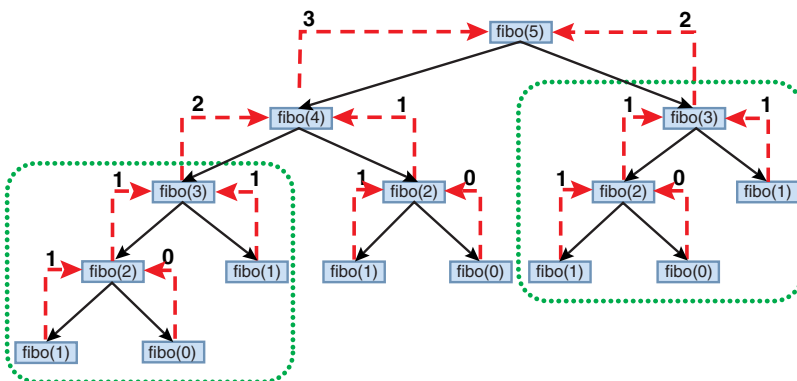


FIGURA 9.34

Por esse motivo, deve-se ter muito cuidado com o uso da recursão na programação.



Recursão é realmente um tópico difícil de entender em programação. Para compreender melhor, leia a Seção 9.3.

9.4 EXERCÍCIOS

9.4.1 Passagem por valor

- 1) Escreva uma função que receba por parâmetro dois números e retorne o maior deles.
- 2) Faça uma função que receba um número inteiro de 1 a 12 e imprima em tela o mês e a sua quantidade de dias de acordo com o número digitado pelo usuário. Exemplo: Entrada = 4. Saída = abril.
- 3) Escreva uma função que receba por parâmetro uma temperatura em graus Fahrenheit e a retorne convertida em graus Celsius. A fórmula de conversão é: $C = (F - 32.0) * (5.0/9.0)$, sendo F a temperatura em Fahrenheit e C a temperatura em Celsius.
- 4) Escreva uma função que receba por parâmetro a altura e o raio de um cilindro circular e retorne o volume desse cilindro. O volume de um cilindro circular é calculado por meio da seguinte fórmula:

$$V = \pi * raio^2 * altura,$$

em que $\pi = 3.1414592$

- 5) Escreva uma função para o cálculo do volume de uma esfera

$$V = 4/3\pi * r^3,$$

em que $\pi = 3.1414592$ valor do raio r deve ser passado por parâmetro.

- 6) Escreva uma função que receba o peso (quilos) e a altura (metros) de uma pessoa. Calcule e retorne o IMC (índice de massa corporal) dessa pessoa:

$$IMC = peso / (altura * altura)$$

- 7) Elabore uma função que receba três números inteiros como parâmetro, representando horas, minutos e segundos. A função deve retornar esse horário convertido em segundos.
- 8) Elabore uma função para verificar se um número é um quadrado perfeito. Um quadrado perfeito é um número inteiro não negativo que pode ser expresso como o quadrado de outro número inteiro. Exemplos: 1, 4, 9.
- 9) Elabore uma função que receba três notas de um aluno como parâmetros e uma letra. Se a letra for “A”, a função deverá calcular a média aritmética das notas do aluno; se for “P”, deverá calcular a média ponderada, com pesos 5, 3 e 2. Retorne a média calculada para o programa principal.
- 10) Escreva uma função que receba dois valores numéricos e um símbolo. Esse símbolo representará a operação que se deseja efetuar com os números. Assim, se o

símbolo for “+”, deverá ser realizada uma adição, se for “-”, uma subtração, se for “/”, uma divisão, e, se for “*”, será efetuada uma multiplicação. Retorne o resultado da operação para o programa principal.

- 11) Escreva uma função que receba por parâmetros dois valores inteiros x e y e calcule e retorne o resultado de x^y para o programa principal. Não use nenhuma função pronta para isso.
- 12) Escreva uma função que receba um número inteiro positivo e retorne o maior fator primo desse número.
- 13) Escreva uma função que receba um número inteiro positivo n . Calcule e retorne o somatório de 1 até n : $1 + 2 + 3 + \dots + n$.
- 14) Escreva uma função que receba um número inteiro positivo n . Calcule e retorne o seu fatorial $n!$: $n! = n * (n - 1) * (n - 2) * \dots * 1$.
- 15) Elabore uma função que receba como parâmetro um valor inteiro n e gere como saída n linhas com pontos de exclamação, conforme o exemplo a seguir, em que usamos $n = 5$:

```
!
!!
!!!
!!!!
!!!!!
```

- 16) Elabore uma função que receba como parâmetro um valor inteiro n e gere como saída um triângulo lateral formado por asteriscos conforme o exemplo a seguir, em que usamos $n = 4$:

```
*
**
***
****
***
**
*
```

- 17) Faça uma função que receba um inteiro N como parâmetro. Calcule e retorne o resultado da seguinte série S :

$$S = \frac{2}{4} + \frac{5}{5} + \frac{10}{6} + \dots + \frac{N^2 + 1}{N + 3}$$

- 18) Faça uma função que receba como parâmetro o valor de um ângulo em graus e calcule o valor do seno desse ângulo usando a sua respectiva série de Taylor:

$$\text{sen } x = \sum_{n=0}^5 \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} \dots$$

em que x é o valor do ângulo em radianos. Considere $\pi = 3.1414592$ e n variando de 0 até 5.

- 19) Faça uma função que receba como parâmetro o valor de um ângulo em graus e calcule o valor do cosseno desse ângulo usando a sua respectiva série de Taylor:

$$\cos x = \sum_{n=0}^5 \frac{(-1)^n}{(2n)!} x^{2n} = x - \frac{x^2}{2!} + \frac{x^4}{4!} \dots$$

em que x é o valor do ângulo em radianos. Considerar $\pi = 3.1414592$ e n variando de 0 até 5.

- 20) Faça uma função que calcule e retorne o número neperiano e , $e = 2,71828183$, usando a série a seguir:

$$e = \sum_{n=0}^N \frac{1}{n!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} \dots$$

A função deve ter como parâmetro o número de termos que serão somados, N . Note que quanto maior esse número, mais próxima do valor e estará a resposta.

9.4.2 Passagem por referência

- 1) Escreva uma função que, dado um número real passado como parâmetro, retorne a parte inteira e a parte fracionária desse número por referência.
- 2) Escreva uma função para o cálculo do volume e área de uma esfera

$$V = \frac{4}{3} \pi r^3,$$

$$A = 4 \pi r^2$$

em que $\pi = 3.1414592$. O valor do raio r deve ser passado por parâmetro, e os valores calculados devem ser retornados por referência.

- 3) Escreva uma função que receba um array de 10 elementos e retorne a sua soma.
- 4) Escreva uma função que receba um array contendo a nota de 10 alunos e retorne a média dos alunos.

- 5) Escreva uma função que calcule o desvio-padrão d de um vetor V contendo n números

$$d = \sqrt{\frac{1}{n-1} \sum_{i=0}^{n-1} (V[i] - m)^2}$$

em que m é a média desse vetor.

- 6) Crie uma função que receba uma matriz A contendo cinco linhas e cinco colunas. Calcule na própria matriz A a sua transposta (se B é a matriz transposta de A , então $A[i][j] = B[j][i]$).
- 7) Crie uma função que receba uma matriz A contendo 10 linhas e 10 colunas e retorne a soma dos seus elementos.
- 8) Faça uma função que receba, por parâmetro, uma matriz A contendo seis linhas e seis colunas. Essa função deve retornar, por referência, a soma dos elementos da sua diagonal principal e da sua diagonal secundária.
- 9) Crie uma estrutura representando um aluno de uma disciplina. Essa estrutura deve conter o número de matrícula do aluno, seu nome e as notas de três provas. Agora, escreva uma função que receba um vetor de tamanho N dessa estrutura. Essa função deve retornar o índice do aluno que possui a maior média geral entre todos os alunos.
- 10) Escreva uma função que receba uma string e retorne se ela é um palíndromo (1) ou não (0). Um palíndromo é uma palavra que tem a propriedade de poder ser lida tanto da direita para a esquerda como da esquerda para a direita. Exemplos: ovo, arara, rever, asa, osso etc.
- 11) Escreva uma função que receba uma string e converta todos os seus caracteres em maiúscula. Dica: subtraia 32 dos caracteres cujo código ASCII está entre 97 e 122.
- 12) Escreva uma função que receba como parâmetro um vetor contendo N valores inteiros. Essa função deve retornar, por referência, dois valores: a soma dos números pares e ímpares.
- 13) Crie uma função que receba um vetor de tamanho N e ordene os seus valores.
- 14) Elabore uma função que receba por parâmetros os coeficientes de uma equação do segundo grau. Em seguida, calcule e mostre as raízes dessa equação. Lembre-se de que as raízes são calculadas como

$$x = \frac{-b \pm \sqrt{\Delta}}{2 * a}$$

em que $\Delta = b^2 - 4 * a * c$ e $ax^2 + bx + c = 0$ representa uma equação do segundo grau. A variável a tem de ser diferente de zero. Caso seja igual, imprima a

mensagem “Não é equação de segundo grau” e retorne o valor -1. Do contrário, retorne o número de raízes e as raízes (por referência) se elas existirem:

- Se $\Delta < 0$, não existe real. Número de raízes: 0.
- Se $\Delta = 0$, existe uma raiz real. Número de raízes: 1.
- Se $\Delta > 0$, existem duas raízes reais. Número de raízes: 2

- 15) Elabore uma função que receba um vetor contendo N valores e retorne por referência o maior e o menor elemento desse vetor.
- 16) Elabore uma função que receba um vetor contendo N valores e retorne por referência o maior elemento do vetor e o número de vezes que esse elemento ocorreu no vetor.

9.4.3 Recursão

- 1) Escreva uma função recursiva que calcule a soma dos primeiros n cubos:

$$S = 1^3 + 2^3 + \dots + n^3$$

- 2) Crie uma função recursiva que receba um número inteiro N e retorne o somatório dos números de 1 a N.
- 3) Crie uma função recursiva que receba um número inteiro N e imprima todos os números naturais de 0 até N em ordem crescente.
- 4) Crie uma função recursiva que receba um número inteiro N e imprima todos os números naturais de 0 até N em ordem decrescente.
- 5) Crie uma função recursiva que retorne a soma dos elementos de um vetor de inteiros.
- 6) Crie uma função recursiva que retorne a média dos elementos de um vetor de inteiros.
- 7) Escreva uma função recursiva que receba por parâmetro dois valores inteiros x e y e calcule e retorne o resultado de x^y para o programa principal.
- 8) A multiplicação de dois números inteiros pode ser feita através de somas sucessivas (por exemplo, $2 * 3 = 2 + 2 + 2$). Crie uma função recursiva que calcule a multiplicação por somas sucessivas de dois inteiros.
- 9) Escreva uma função recursiva que receba um número inteiro positivo n . Calcule e retorne o seu fatorial $n!$:

$$n! = n * (n - 1) * (n - 2) * \dots * 1$$

- 10) Escreva uma função recursiva que receba um número inteiro, maior ou igual a zero, e retorne o n ésimo termo da sequência de Fibonacci. Essa sequência começa

no termo de ordem zero e, a partir do segundo termo, seu valor é dado pela soma dos dois termos anteriores. Alguns termos dessa sequência são: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

- 11) Escreva uma função recursiva que receba um valor inteiro e o retorne invertido.
Exemplo:

Número lido = 123.

Número retornado = 321.

- 12) Escreva uma função recursiva que receba um valor inteiro em base decimal e o imprima em base binária.
- 13) Faça uma função recursiva que calcule o valor da série S descrita a seguir para um valor n maior do que zero a ser fornecido como parâmetro para a mesma:

$$S = 2 + \frac{5}{2} + \frac{10}{3} + \dots + \frac{1+n^2}{n}$$

- 14) Crie uma função recursiva que retorne o menor elemento em um vetor.
- 15) Em matemática, o número harmônico designado por H_n define-se como o n -ésimo termo da série harmônica. Ou seja:

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n}$$

Escreva uma função recursiva que calcule o valor de qualquer H_n .